

Final Project Report: Mini AMR ROS 2 Simulation Platform

Project Title:

ROS 2 Based Mini Autonomous Mobile Robot (AMR) Simulation and Navigation Platform

Development Environment:

Ubuntu 24.04 LTS (WSL2)

ROS 2 Jazzy Jalisco

Project Period:

June 9, 2026 – August 6, 2026

This report presents the development of a ROS 2 based Mini-Autonomous Mobile Robot (AMR) simulation platform. The objective of this project is to study and implement the fundamental technologies required for autonomous mobile robots, including robot modeling, motion control, sensor processing, SLAM mapping, and autonomous navigation.

The system was developed using ROS 2 Jazzy on Ubuntu 24.04 and consists of five ROS 2 packages responsible for robot description, control, sensing, navigation, and system integration. A simulated LiDAR sensor and odometry system were implemented to enable mapping and autonomous navigation without physical hardware.

The final system successfully performed map-based autonomous navigation using the ROS 2 Navigation2 framework. Experimental results showed that the robot reached the target position with a position error of approximately 4.2 cm, demonstrating the effectiveness of the developed Mini-AMR platform.

1. Introduction

Autonomous Mobile Robots (AMRs) are widely used in modern smart manufacturing and warehouse automation systems for material transportation and autonomous operation. This project focuses on developing a ROS 2 based Mini-AMR simulation platform to study the fundamental components of an autonomous mobile robot, including robot modeling, motion control, sensor processing, mapping, and autonomous navigation.

The objective of this project is to develop a complete software-based AMR system without requiring physical robot hardware. The developed platform includes a robot model created using URDF/Xacro, velocity control through ROS 2 communication interfaces, simulated LiDAR sensing, SLAM mapping, and autonomous navigation using the Navigation2 (Nav2) framework.

The project was developed incrementally, starting from robot description and coordinate transformation, followed by motion control, sensor processing, SLAM mapping, and final autonomous navigation testing.

The final system successfully demonstrated autonomous navigation to a target position using ROS 2 Nav2 with a measured position error of approximately 4.2 cm.

2. System Architecture

The Mini-AMR platform consists of five main ROS 2 packages:

Package	Description
mini_amr_description	Robot model, URDF/Xacro, and RViz visualization
mini_amr_control	Odometry calculation and motion control modules
mini_amr_sensors	LiDAR simulation and safety monitoring
mini_amr_navigation	SLAM, map management, and Nav2 navigation
mini_amr_bringup	System launch configuration

2.1 Overall System Architecture

The overall system architecture of the Mini-AMR platform is designed based on the ROS 2 framework, integrating robot modeling, motion control, sensor processing, SLAM mapping, and autonomous navigation modules.

The system is divided into several functional layers. The control layer receives velocity commands from either teleoperation or the Nav2 controller through the `/cmd_vel` interface. The motion control and odometry modules process these commands to estimate robot movement and publish odometry information through the `/odom` topic and TF transformation system.

The perception layer provides simulated LiDAR data through the `/scan` topic, which is used for environmental sensing and mapping. SLAM Toolbox is responsible for generating the occupancy grid map, while the Nav2 framework performs localization, path planning, and autonomous navigation based on the generated map.

The complete communication flow allows the Mini-AMR to move from manual control operation to fully autonomous navigation, demonstrating the fundamental workflow of an AMR system.

2.2 TF Coordinate Structure

The TF (Transform) coordinate structure was designed to represent the spatial relationship between different components of the Mini-AMR robot. In ROS 2, TF is responsible for maintaining the position and orientation relationship between robot links, sensors, and the global environment.

The designed TF tree consists of the global map frame, robot odometry frame, robot base frames, and sensor frames. This structure allows the robot to estimate its position, visualize sensor data, perform SLAM mapping, and execute autonomous navigation.

The TF coordinate structure is shown below:

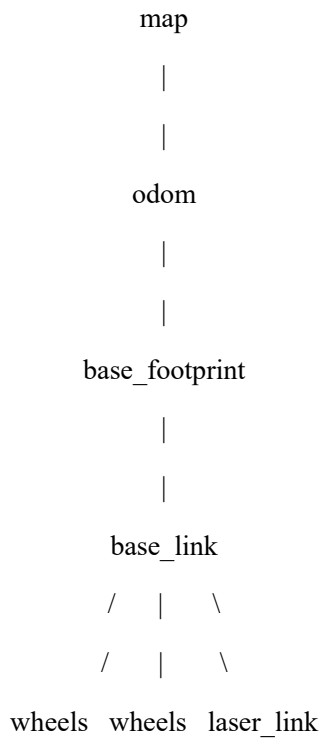


Figure 1. TF coordinate structure of the Mini-AMR ROS 2 system

Frame	Description
map	Global reference frame generated by SLAM or used by Nav2 localization
odom	Local odometry frame representing short-term robot movement estimation
base_footprint	Ground projection frame used as the robot reference position
base_link	Main robot body frame defined in the URDF model
laser_link	LiDAR sensor frame used for laser scan data transformation
Wheel frames	Individual wheel links for robot visualization and joint movement

The transformation relationship between frames is managed through ROS 2 TF publishers:

- odom → base_footprint is provided by the odometry node to represent robot movement.
- base_footprint → base_link is defined by the robot URDF model.
- base_link → laser_link allows LiDAR data to be correctly transformed into the robot coordinate system.
- map → odom is generated by SLAM Toolbox during mapping or AMCL during autonomous navigation.

The main ROS 2 communication interfaces include:

Topic / Interface	Function
/cmd_vel	Robot velocity command
/odom	Odometry estimation
/scan	LiDAR sensor data
/map	Occupancy grid map
/initialpose	Robot initial localization
/navigate_to_pose	Nav2 navigation action
/safety_markers	Safety zone visualization

3. Implementation

3.1 Robot Model Development

The robot model was created using URDF/Xacro. The model consists of:

- A main robot body (base_link)
- Four wheel components
- A LiDAR sensor frame (laser_link)
- Joint definitions for robot movement visualization

The robot description package provides visualization capability through RViz2 and publishes the required TF transformations using ROS 2 robot state tools.

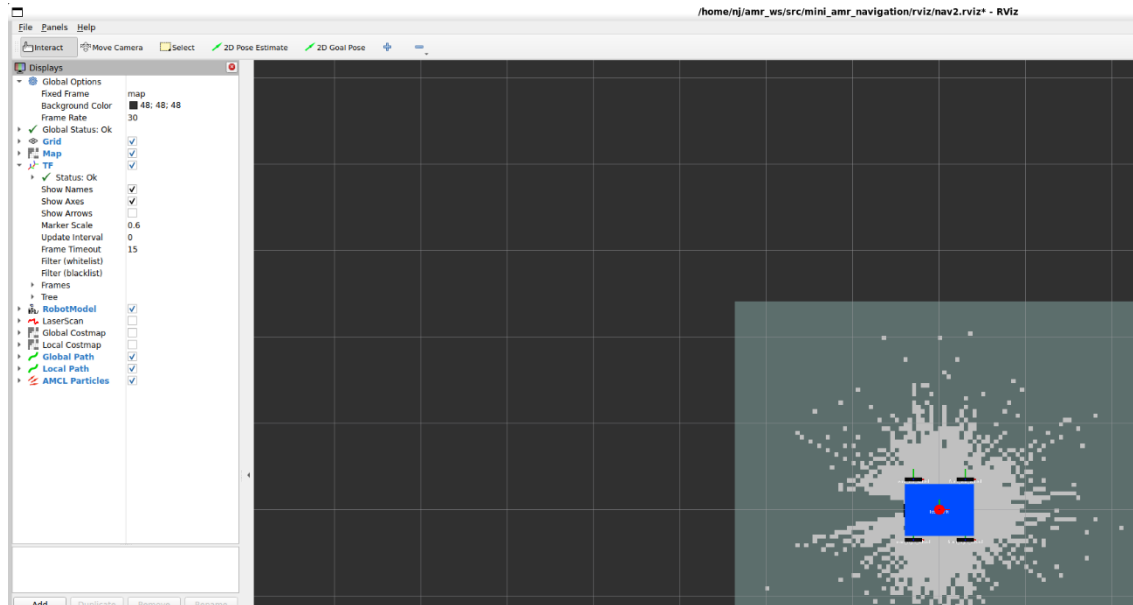


Figure 2. Mini-AMR robot model visualization in RViz2

3.2 Motion Control and Odometry

The robot motion system was developed using ROS 2 velocity commands.

The `/cmd_vel` topic is used as the main control interface, where linear and angular velocities are transmitted to the robot control system.

An odometry node was implemented to estimate robot position and orientation by integrating velocity commands over time. The node publishes:

- `/odom` topic
- `odom` → `base_footprint` TF transformation

This provides the localization information required by SLAM and Nav2.

A mecanum wheel kinematic model was also developed to analyze omni-directional robot motion and calculate individual wheel velocities. This module was developed as a motion analysis component during the control development stage.

3.3 LiDAR Processing and Safety Monitoring

A simulated 2D LiDAR sensor was developed to provide LaserScan data through the `/scan` topic.

The sensor module includes:

- 360-degree scan simulation
- Front, left, and right obstacle detection
- Safety zone visualization

The obstacle detection node analyzes minimum distances from different directions and publishes obstacle warning information.

The safety visualization module displays the robot safety area in RViz2 using marker visualization.

3.4 SLAM Mapping

SLAM (Simultaneous Localization and Mapping) was implemented using SLAM Toolbox.

The robot collected LiDAR data while moving through the environment and generated an occupancy grid map.

The mapping process included:

1. Robot movement using teleoperation
2. LiDAR data collection
3. SLAM processing
4. Map saving using ROS 2 map server tools

The final generated map was prepared for autonomous navigation.

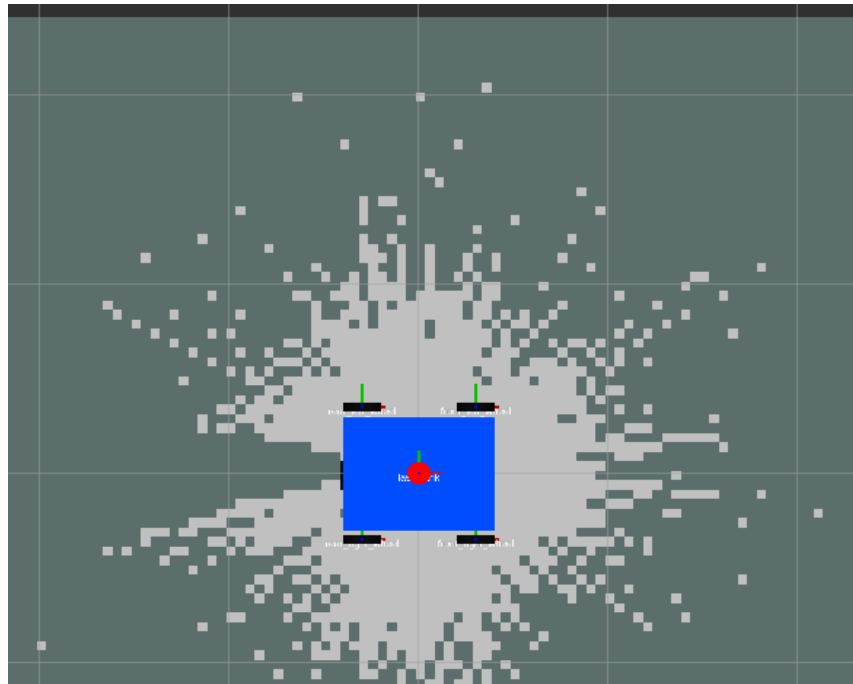


Figure 3. Occupancy grid map generated using SLAM Toolbox

3.5 Autonomous Navigation Using Nav2

The Navigation2 framework was integrated for autonomous navigation.

The navigation system includes:

- Map server
- AMCL localization
- Global planner
- Local controller
- Behavior tree navigator

The robot receives a navigation goal through RViz2 and calculates a collision-free path using the generated map.

4. Experimental Results

The final navigation experiment was performed using the generated map and Nav2 framework.

The robot successfully reached the target position.

4.1 Navigation Performance

Parameter	Result
Target position	(1.00, 0.80) m
Final robot position	(1.040, 0.789) m
Position error	0.042 m (4.2 cm)
Navigation status	SUCCESS

The navigation action result confirmed:

- Goal succeeded
- Reached the goal

The final demonstration video is included in the project deliverables: File name: mini_amr_final_demo.mp4

The video shows the complete Mini-AMR workflow, including:

- Robot model visualization
- Teleoperation movement
- SLAM mapping process
- Nav2 autonomous navigation

The final demo duration is 60 seconds with 1280×720 resolution.

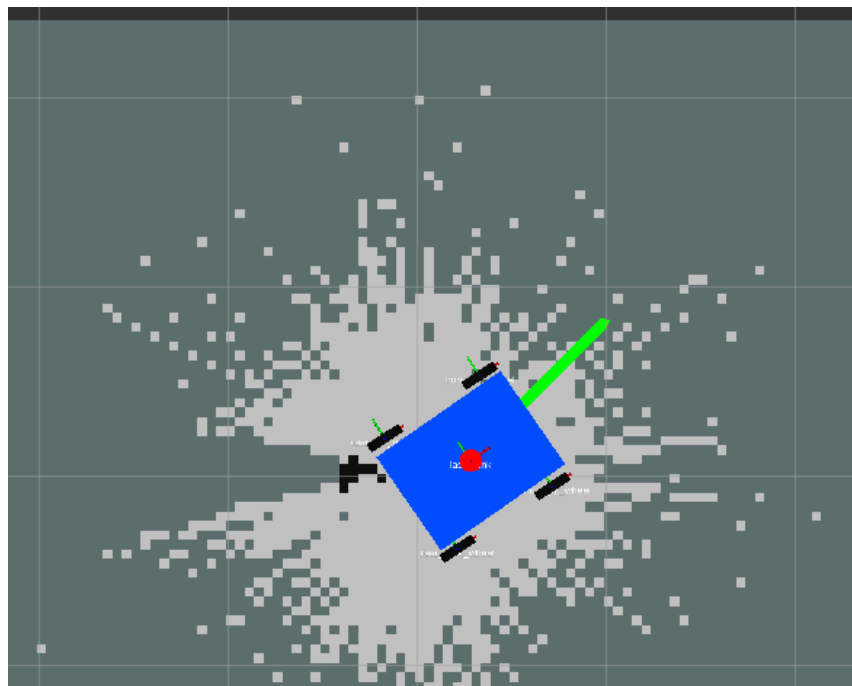


Figure 4. Autonomous navigation result using the Nav2 framework.

5. Problems and Solutions

During development, several technical issues were encountered and resolved.

Problem	Solution
TF conflict caused incorrect robot movement visualization	Removed duplicated TF broadcasting and corrected TF structure
SLAM node did not start correctly	Added lifecycle management for SLAM operation
Generated map contained noise	Applied map refinement and denoising process
Hardcoded file paths reduced portability	Replaced paths with ROS package-based file locations
Navigation instability during testing	Adjusted navigation parameters and map configuration

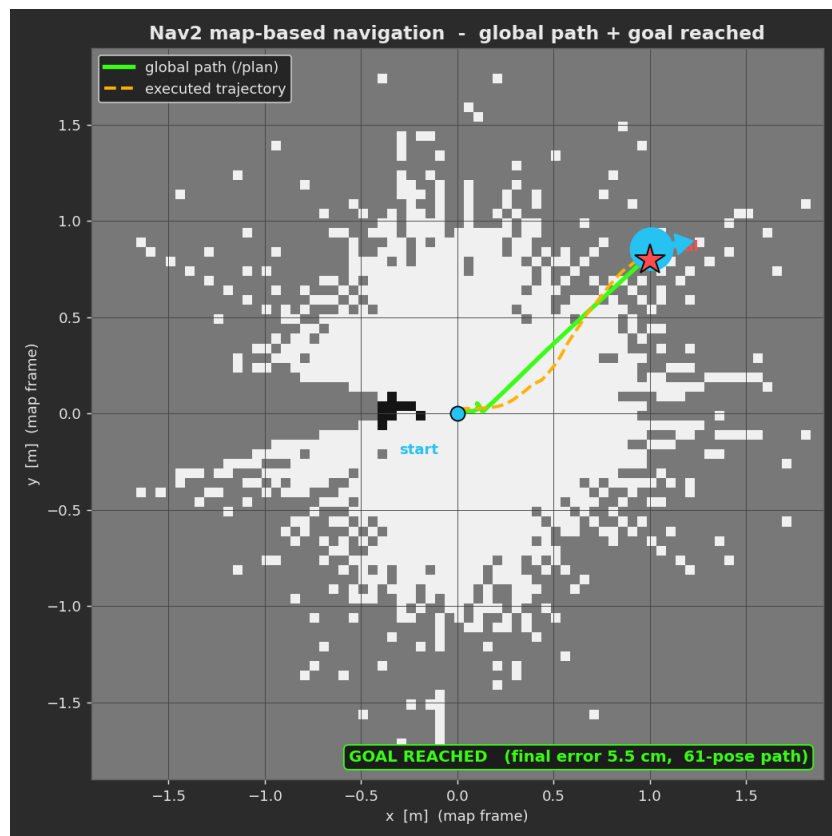


Figure 5. Robot trajectory and navigation performance

6. Future Improvements

Although the simulation platform achieved autonomous navigation, several improvements can be implemented in future development.

6.1 Hardware Integration

The current system uses simulated odometry and LiDAR data. Future work can integrate:

- Real LiDAR sensors
- Motor drivers
- Wheel encoders
- Embedded controllers

6.2 Improved Simulation Environment

A Gazebo based simulation environment can be added to provide:

- Realistic robot physics
- Dynamic obstacles
- Sensor simulation

6.3 Advanced AMR Functions

Future improvements include:

- Dynamic obstacle avoidance
- Multi-AMR coordination
- Warehouse environment simulation
- Fleet management system integration

7. Conclusion

This project successfully developed a ROS 2 based Mini-AMR simulation platform covering the complete autonomous robot development workflow. The system includes robot modeling, motion control, sensor processing, SLAM mapping, and autonomous navigation using Nav2. The final experiment demonstrated successful goal-based navigation with a position error of only 4.2 cm. This result proves that the developed platform can serve as a foundation for further AMR research and future hardware implementation. The developed platform provides a complete learning environment for understanding AMR software architecture and can be further extended toward real warehouse automation applications.